Linux Capabilities

Computing Systems Security · MCS and MEI · University of Minho · 2026-27

Vítor Francisco Fonte <vff@di.uminho.pt> · 2026-08-19

Capabilities: Lecture Overview

The Problem Capabilities Solve: why a program that needs one power should not be given all of root

File Capabilities: attaching a single power to a binary, and why this is harder to adopt than it sounds

Capabilities and Containers: the default set, the privileged set, and the three capabilities that matter most

Into the Lab: reading capabilities on a real system, and what dropping one does not buy you

The tab on the right edge of each slide shows the tier of the slide.

Lecture Core: covered in session. Lab Core: connects to laboratory work. Study Support: post-session review. Reference: lookup material.

One Power, Not Forty

- Last week's complaint, restated: setuid grants **all** of root, for the life of the process
- `ping` opens a raw socket. A web server binds port 80. A backup tool reads files it does not own
- Each of those is **one** power. Under setuid each of them receives every power root has
- So the gap between grant and need is not a rounding error. It is the entire difference

The mechanism was never too weak. It was always too coarse.

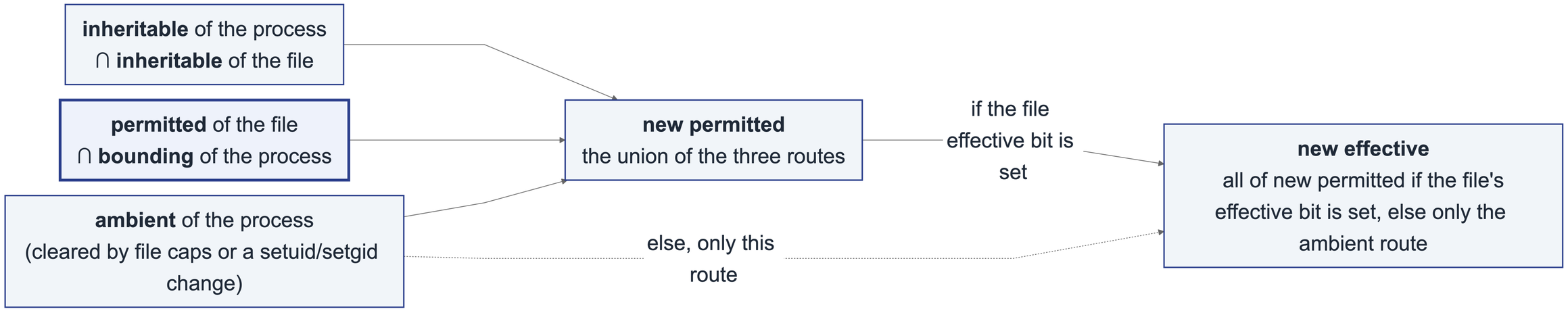
Splitting root Into Pieces

- Linux splits root's authority into about **forty independent capabilities**, each naming one privileged class
- `CAP_NET_BIND_SERVICE`: bind port 80. `CAP_CHOWN`: change ownership. `CAP_SYS_MODULE`: load kernel code
- A process holds a **set** of them, and each privileged syscall checks for the one it needs
- The idea is general, not a Linux invention: privilege separation appears in Solaris, BSD and Windows too

Root stops being a switch and becomes a list. Everything today follows from that one change.

The Five Sets

- **Permitted** is what a process may hold; **effective**, what it uses now; **bounding** masks only the file's grant (thick box)
- **Inheritable** counts via the file's inheritable set; **ambient** carries over on an ordinary `exec`
- The diagram is for a non-root process: for root, the kernel treats the file's sets as full



Permitted is what you may pick up; effective is what is in your hand; bounding is the mask the next `exec` applies.

Effective Now, Bounding at Exec

```
$ grep Cap /proc/self/status
CapEff: 0000000000000000
CapBnd: 000001ffffffffff

$ capsh --decode=0000000000000000
0x0000000000000000=

# a daemon's effective set is not empty
```

- **`CapEff`** is what the kernel checks on a privileged syscall, and it is empty for your shell
- **`CapBnd`** is the mask your next `exec` will apply, and yours is full: it masks nothing
- An empty effective set with a full bounding set is not a sandbox, only nothing acquired yet
- `capsh --decode` turns the bitmask into names, which is the only humane way to read these

Read both. The effective set is what is active now; bounding is the mask the next `exec` applies.

Lab Core

File Capabilities

If a program needs one power, can we attach that one power to the program?

Attaching One Power to a Binary

```
$ sudo setcap cap_net_bind_service+ep ./capbind
$ getcap ./capbind
./capbind cap_net_bind_service=ep

# e = effective, p = permitted
```

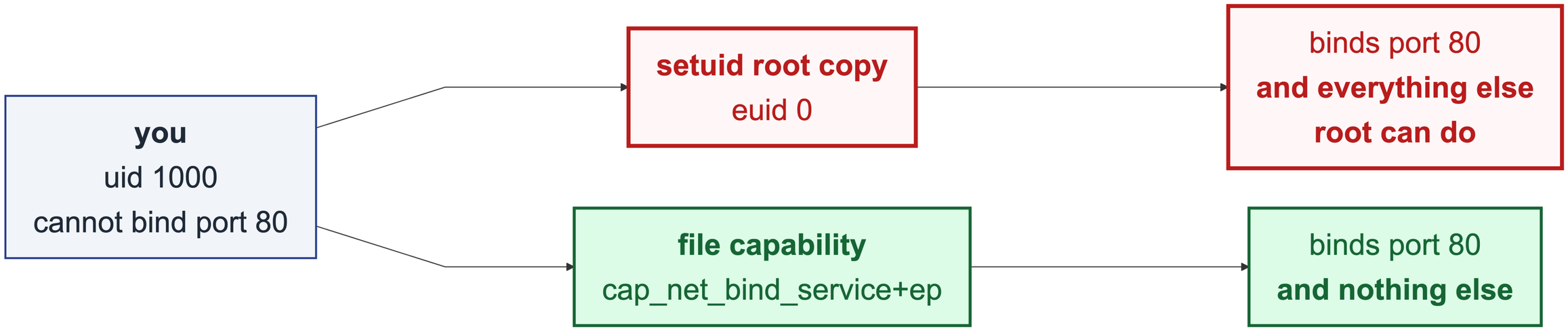
- The capability lives on the **file**, where the setuid bit lives, with a fraction of the grant
- `+ep` says: put it in the permitted set on `exec`, and make it effective immediately
- Removal is another `setcap` flag; the lab asks you to find it. Both need `CAP_SETFCAP`: hence `sudo`
- This is the direct replacement for a setuid binary that needed one power, and it is one line

Lab Core

Same place, same idea, smaller grant. The mechanism is not the obstacle to least privilege here.

The Blast Radius, Side by Side

- Same program, same job, two grants. You measure it in Lab 7, on your own machine
- The setuid copy binds the port **as euid 0**: for its whole run it can do anything root can
- The file-capability copy binds the port **as you**, holding one power and no identity change at all



This is why ping is no longer setuid root on modern Linux. The same fix, applied years ago.

Reading Them on a Running System

```
$ getcap -r /usr/bin 2>/dev/null
/usr/bin/ping cap_net_raw=ep

$ grep CapEff /proc/$(pgrep -n cron)/status
CapEff: 000001ffffffffff

$ capsh --decode=000001ffffffffff
0x000001ffffffffff=cap_chown,cap_dac_override,...
```

- `getcap -r` sweeps a tree for binaries carrying capabilities, beside `find -perm -4000`
- A daemon started by `init` typically holds everything, which is what a privileged service is
- `capsh --decode` is how you turn any of these bitmasks into names you can argue about
- Three commands, and you can audit the privilege posture of a machine you have never seen

Run getcap beside last week's setuid sweep in your first ten minutes on a host. A capability is as interesting as a setuid bit and less looked at.

Lab Core

Last Week's backup-tool, Revisited

- The seeded tool was a setuid copy of `cp`: it could read and write **any** file, as root, for whoever ran it
- What it needed was to read any file, whatever its mode bits, which is `CAP_DAC_READ_SEARCH`
- The obvious fix: remove the setuid bit and grant only that capability. You tested it in Lab 6, Exercise 5
- The read of `/etc/shadow` still works under that one capability: the grant shrank, the hole did not close

A smaller grant shrinks the blast radius. Closing the hole means removing the privilege, limiting what it touches, or who can run it.

Why This Is Harder Than It Sounds

- Most privileged programs were written assuming full root, and fail in obscure ways when they hold less
- Finding the minimal set is real work: you discover it by running the thing and watching what breaks
- The five sets interact, and a change that looks right can be silently ineffective
- Which is why setuid persists in 2026, on systems whose authors know all of this

The idea is twenty-five years old and adoption is still partial. Assume the mechanism is right and the practice is hard.

Ambient, and Why a Drop Can Fail

- **Inheritable** bits reach the new permitted set only where the file's inheritable set also has them; alone they do nothing
- The **ambient** set was added to fix that: a capability in it crosses `exec` into programs with no file capabilities
- Non-root, two traps: a plain re-exec loses what is not ambient; a bounding drop does not stop an ambient copy
- The rules are documented and unintuitive. Read `capabilities(7)` before designing around them

Study Support

If a capability change did not do what you expected, the answer is almost always in which set you changed.

Capabilities and Containers

Your container runs as root. How much of root is that, exactly?

Not a Capability Boundary by Itself

- A container is namespaces, cgroups, a capability set, a syscall filter and a policy. Capabilities are one part
- Docker keeps about **fourteen** of the forty by default, which is a real reduction and not a sandbox
- The container's `root` is therefore a diminished root, and the size of the diminishment is a configuration choice
- Reasoning about containers without reading that set is guessing

Container root is not host root, and it is not an ordinary user either. It is whatever the set says.

Default and Privileged, Measured

```
# a default container
$ docker run --rm alpine \
    grep CapEff /proc/self/status
CapEff: 00000000a80425fb

# with --privileged
CapEff: 000001fffffffffff
```

- `a80425fb` is the default set on both arm64 and amd64 images: it is architecture-neutral
- All ones is what `--privileged` means. Not 'more privileged': **every capability the kernel has**
- So `--privileged` is not a debugging convenience with a security cost. It is host root with extra steps
- Decode both in the lab and read what the fourteen actually are

One flag turns a diminished root into a complete one. It is the most consequential flag in Docker.

Lab Core

Three That Matter

- **`CAP_SYS_ADMIN`** is called the new root for a reason: mounting, namespace operations, and a long tail of others
- **`CAP_SYS_MODULE`** loads kernel code. A process with it defines the kernel, so nothing above constrains it
- **`CAP_SYS_PTRACE`** sounds like a debugger's permission. Where the attach is allowed, it rewrites another process
- Any of the three, granted to a container, is close to the end of the argument about isolation
- Which is why the interesting question is never how many capabilities, but **which**

Counting capabilities is not measuring privilege. Three of the forty are worth more than the other thirty-seven.

Non-Root Is Not Sandboxed

- Running a container process as a non-root UID is good practice and is not, by itself, containment
- The capability set is a property of the **process**, and a non-root UID with a fat set is still dangerous
- Conversely a root UID with `--cap-drop ALL` may be safer than a non-root one with the defaults
- The two controls are independent, and 'runs as non-root' on a report page tells you about one of them

Ask two questions, not one: which UID, and which capabilities. Either alone is half an answer.

Start From Nothing

```
$ docker run --rm \
    --cap-drop ALL \
    --cap-add NET_BIND_SERVICE \
    myimage

# not: --cap-drop SYS_ADMIN and hope
```

- Drop everything, then add back what the workload needs: a decision, not an inheritance
- Trimming the default set instead means you must know all fourteen, and be right about every one
- The construction is the same least-privilege move as `(root) NOPASSWD: one command` last week
- And the same cost: you find the minimum by running the thing and watching it fail

Start from nothing and justify each addition. Starting from the default and subtracting is how capabilities are kept by accident.

The Configuration Trap

```
# inside a default container
$ sysctl net.ipv4.ip_unprivileged_port_start
net.ipv4.ip_unprivileged_port_start = 0

# so binding :80 needs no capability at all
# the gate you reasoned about is not in play
```

- Docker sets that sysctl to 0, which removes the privileged-port reservation entirely
- A container binds port 80 with **no** capability, so `NET_BIND_SERVICE` describes nothing here
- The mechanism was not bypassed. The configuration moved the boundary it was guarding
- Reasoning from capabilities alone, without reading the surrounding configuration, is incomplete

A control you reasoned about correctly can be irrelevant. Check whether the gate is still where you think it is.

Lab Core

Not Containment, Yet

- You will be tempted to conclude that `CAP_SYS_ADMIN` alone breaks a container. Right instinct, wrong time
- The full analysis needs namespaces (Week 9) and syscall filtering (Week 10), which you do not have yet
- Week 12 measures it properly: which layer refuses first, and what a granted capability actually unlocks
- The answer, when it comes, is more interesting than either version you would guess today

The question is parked, not dodged. Week 12 measures it on your own machine, one layer at a time.

In the Laboratory

Part A only reads. Clear the file capability with sudo at the end: it needs CAP_SETFCAP, not ownership.

- Read the five `Cap` lines of your shell and of a daemon; decode both `CapEff` and the shell's `CapBnd`
- Build the bind helper, `lab7-bind80.c` from the zip, and compare its setuid and file-capability copies
- Read the default container set, then the privileged one, and decode the difference
- Then find the minimal grant for a service, which is the part that takes the longest

Lab Core

Decode every bitmask you meet in the lab. A capability set you have not decoded is a number, not an argument.

Dropped Is Not Prevented

- Today's through-line: 'I dropped the dangerous capability' is not 'the dangerous action is prevented'
- `--privileged` puts everything back with one flag, and a sysctl can move the boundary the capability guarded
- A control is only as good as the premises around it, and those premises are configuration
- So state the premise when you state the control. That habit is what the capstone will test

Name the control, then name what it assumes. The second half is where the findings are.

Reading and Reference

- **capabilities(7)**. The reference for all five sets and the inheritance rules. The one page worth reading in full.
  https://www.man7.org/linux/man-pages/man7/capabilities.7.html
- **setcap(8), getcap(8), capsh(1)**. The tools from libcap, all three used in Lab 7.
- **Docker, Runtime privilege and Linux capabilities**. The default set, and what --privileged actually does.
  https://docs.docker.com/engine/containers/run/
- **Saltzer and Schroeder, 1975**. Least privilege again: capabilities are the clearest implementation of it in this unit.
  https://www.cs.virginia.edu/~evans/cs551/saltzer/
- **W. Du, Computer Security: A Hands-on Approach, 3rd ed., 2022**. The capability and container chapters.

Reference

Key Takeaways

- Root is a list, not a switch: about forty capabilities, checked one syscall at a time
- Five sets, and most confusion is about which one a question concerns. Effective now, bounding at exec
- A file capability does a setuid binary's job with a fraction of the grant, which is why `ping` moved
- In containers: read the set. `--privileged` is all of it, and the three from earlier are worth more than the rest
- **Dropped is not prevented.** State the control and the configuration it assumes

Which UID, which capabilities, and what does the surrounding configuration assume? Three questions, one posture.